

MAKALAH
HISTOGRAM CITRA

Untuk Memenuhi Tugas Mata Kuliah Pengolahan Citra Digital
Yang Diampu Oleh

Mohamad Yasin, S.Kom., M.Kom.



Disusun oleh:

Dana Rayka Mujahidin	230312607384
Nasywa Salsabila	230312605053
Shinta Arum Sari	230312606001

UNIVERSITAS NEGERI MALANG
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
S1 MATEMATIKA
SEPTEMBER 2026

DAFTAR ISI

BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Rumusan Masalah.....	3
1.3 Tujuan	4
BAB II.....	5
ISI.....	5
2.1 Citra Digital dan Representasi Piksel	5
2.2 Histogram.....	5
2.3 Perhitungan Histogram	6
2.4 Plot Histogram	8
2.5 Histogram Stretching	8
2.6 Histogram Equalization	10
2.7 Histogram Warna	12
2.8 Pembentukan dan Interpretasi Histogram Warna.....	13
BAB III	15
KESIMPULAN.....	15
3.1 Kesimpulan	15
3.2 Saran	16
DAFTAR PUSTAKA.....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengolahan citra digital (*digital image processing*) merupakan bidang ilmu yang mempelajari cara mengolah citra, baik berupa gambar diam maupun gambar bergerak. Cakupan bidang ini meliputi akuisisi citra, peningkatan kualitas citra (*image enhancement*), perbaikan citra (*image restoration*), transformasi citra, segmentasi, ekstraksi fitur, hingga pengenalan pola (Gonzalez & Woods, 2008). Agar proses peningkatan dan perbaikan kualitas citra dapat dilakukan dengan tepat, diperlukan suatu alat bantu yang mampu menggambarkan karakteristik sebaran intensitas piksel pada citra secara menyeluruh. Alat bantu yang umum digunakan untuk keperluan tersebut adalah histogram citra.

Histogram citra adalah grafik yang menggambarkan sebaran nilai intensitas piksel yang menyusun suatu citra (Oliver, 1998). Pada citra grayscale (keabuan) 8 bit, nilai intensitas piksel berkisar antara 0 sampai 255, di mana nilai 0 mewakili warna hitam dan nilai 255 mewakili warna putih. Histogram dari citra tersebut menampilkan jumlah piksel untuk setiap dari 256 tingkat keabuan yang mungkin muncul, sehingga dari bentuknya dapat diketahui sebaran gelap-terang (*exposure*), tingkat kecerahan (*brightness*), serta kontras suatu citra (Mustafa & Abdul Kader, 2018).

Informasi yang diperoleh dari histogram dapat dimanfaatkan untuk menilai kualitas citra secara kuantitatif, misalnya untuk mengetahui apakah suatu citra terlalu gelap, terlalu terang, atau memiliki kontras yang rendah. Berdasarkan hasil analisis tersebut, dapat dilakukan proses peningkatan kualitas citra, di antaranya melalui teknik *contrast stretching* (histogram stretching) dan histogram equalization (Oliver, 1998; Xiong et al., 2021). Kedua teknik ini bekerja dengan memanfaatkan pemetaan ulang nilai intensitas piksel berdasarkan histogram citra, sehingga kontras citra dapat diperbaiki (Yousuf & Rakib, 2011). Selain pada citra keabuan, konsep histogram juga dapat diterapkan pada citra berwarna melalui histogram warna, yang menggambarkan sebaran intensitas pada masing-masing kanal warna penyusun citra tersebut.

Berdasarkan uraian di atas, makalah ini disusun untuk membahas konsep histogram citra, cara menghitung dan menampilkannya menggunakan bahasa pemrograman Python dengan bantuan pustaka OpenCV dan Numpy, serta teknik peningkatan kualitas citra yang memanfaatkan histogram, yaitu *contrast stretching* dan histogram equalization.

1.2 Rumusan Masalah

1. Apa yang dimaksud dengan histogram pada citra digital?
2. Bagaimana cara menghitung dan menampilkan (*plotting*) histogram suatu citra?
3. Bagaimana histogram dapat digunakan untuk menganalisis *exposure*, *brightness*, dan kontra suatu citra?

4. Bagaimana konsep dan penerapan *contrast stretching* (histogram stretching) pada citra?
5. Bagaimana konsep dan penerapan histogram equalization pada citra?
6. Bagaimana representasi histogram pada citra warna?

1.3 Tujuan

1. Menjelaskan konsep dasar histogram pada citra digital.
2. Menjelaskan cara perhitungan dan plotting histogram menggunakan OpenCV dan Numpy
3. Menjelaskan penggunaan histogram untuk analisis *exposure*, *brightness*, dan kontras citra.
4. Menjelaskan konsep dan implementasi *contrast stretching* atau histogram stretching.
5. Menjelaskan konsep dan implementasi histogram equalization.
6. Menjelaskan representasi histogram pada citra warna (RGB).

BAB II

ISI

2.1 Citra Digital dan Representasi Piksel

Citra digital tersusun atas kumpulan elemen terkecil yang disebut piksel (*pixel*), yang masing-masing memiliki nilai intensitas tertentu. Pada citra grayscale, nilai intensitas piksel umumnya berkisar antara 0 hingga 255 (rentang 8-bit), di mana nilai 0 merepresentasikan warna hitam dan nilai 255 merepresentasikan warna putih. Sedangkan pada citra berwarna, setiap piksel direpresentasikan oleh kombinasi nilai pada beberapa channel warna, misalnya Red, Green, dan Blue (RGB).

2.2 Histogram

Menurut Prasvita dkk. (2023), histogram merupakan representasi grafis dari distribusi data. Dalam pengolahan citra digital, data yang ditampilkan pada histogram adalah persebaran frekuensi *pixel* yang memiliki nilai intensitas (tingkat terang-gelap) yang sama. Histogram merupakan representasi grafis dari distribusi data. Dalam konteks pengolahan citra, histogram menggambarkan persebaran frekuensi kemunculan piksel yang memiliki nilai intensitas yang sama. Setiap nilai intensitas pada histogram direpresentasikan sebagai “bin”. Untuk citra grayscale, jumlah bin yang digunakan biasanya sebanyak 256, karena rentang nilai intensitas berkisar dari 0 sampai 255. Pada grafik histogram, sumbu horizontal (x) menunjukkan nilai intensitas piksel, sedangkan sumbu vertikal (y) menunjukkan jumlah piksel (frekuensi) yang memiliki nilai intensitas tersebut.

Histogram tidak hanya berfungsi sebagai alat visualisasi, tetapi juga dapat dimanfaatkan untuk melakukan image enhancement dan menentukan nilai threshold pada proses segmentasi citra. Selain itu, histogram dapat digunakan untuk menganalisis *exposure*, *brightness*, dan kontras, serta *dynamic range* suatu citra, sebagaimana diuraikan berikut.

a. *Exposure*

Exposure menunjukkan sebaran nilai intensitas piksel di dalam suatu citra. Sebuah citra dapat dikatakan ter-*expose* dengan baik apabila seluruh bin dari ujung kanan hingga ujung kiri histogram terisi tanpa ada ruang yang kosong. Sebaliknya, citra yang underexposed memiliki histogram yang cenderung mengumpul pada bin bernilai rendah (gelap), sedangkan citra yang overexposed memiliki histogram yang cenderung mengumpul pada bin bernilai tinggi (terang).

b. *Brightness*

Brightness (kecerahan) suatu citra dapat diatur dengan cara menggeser posisi histogram. Menggeser histogram ke arah kanan (nilai intensitas lebih tinggi) akan meningkatkan brightness, sedangkan menggesernya ke arah kiri akan menurunkan brightness.

c. Kontras

Kontras suatu citra dapat dilihat dari lebar sebaran histogram. Citra dengan *low contrast* memiliki histogram yang mengumpul pada rentang nilai intensitas yang sempit, citra dengan normal *contrast* memiliki sebaran histogram yang lebih merata, sedangkan citra dengan high *contrast* memiliki histogram yang tersebar hingga menggunakan rentang bin yang lebih luas.

Berdasarkan kondisi *exposure*-nya, citra dapat dikelompokkan menjadi tiga, yaitu citra *underexposed* (gelap), citra *properly exposed* (normal), dan citra *overexposed* (terang). Pada citra *underexposed*, sebaran histogram terkonsentrasi pada nilai-nilai grayscale yang rendah sehingga menumpuk di sisi kiri histogram. Pada citra *overexposed*, sebaliknya, sebaran histogram terkonsentrasi pada nilai-nilai grayscale yang tinggi sehingga menumpuk di sisi kanan histogram. Adapun citra yang normal umumnya memiliki tingkat *brightness* pada rentang tengah, tidak mengalami saturasi intensitas, serta memiliki sebaran histogram yang cukup lebar sehingga kontrasnya relatif tinggi.

2.3 Perhitungan Histogram

Perhitungan histogram dapat dilakukan dengan dua cara, yaitu menggunakan *library* OpenCV dan Numpy. Pada *library* OpenCV, histogram dihitung menggunakan fungsi `cv.calcHist()`, sementara pada Numpy perhitungan histogram menggunakan fungsi `np.histogram()`.

Perhitungan histogram dengan OpenCV:

```
cv.calcHist(images, channels, mask, histSize, ranges)
```

Parameter-parameter tersebut dijelaskan sebagai berikut.

- **Images:** gambar sumber dengan tipe uint8 atau float32, harus diberikan dalam tanda kurung siku, contoh [img].
- **channels:** diberikan dalam tanda kurung siku, merupakan channel tempat menghitung histogram. Untuk gambar skala abu-abu, nilainya adalah [0]. Untuk gambar berwarna, dapat menggunakan [0], [1], atau [2] untuk menghitung histogram masing-masing channel biru, hijau, atau merah.
- **mask:** mask image; untuk menemukan histogram dari gambar secara keseluruhan, diisikan "None".
- **histSize:** mewakili jumlah bin pada histogram, diberikan dalam tanda kurung. Untuk skala penuh digunakan [256].
- **ranges:** rentang nilai intensitas yang dihitung, biasanya [0, 256].

Contoh perhitungan histogram keabuan dengan OpenCV:

```
import cv2 as cv
import matplotlib.pyplot as plt

gambar = cv.imread("IMG_9549.jpg", cv.IMREAD_GRAYSCALE)

his = cv.calcHist([gambar], [0], None, [256], [0,256])
```

```
his = his.flatten() # pastikan jadi array 1 dimensi

for i in range(0, 256, 20):
    print(f"Intensitas {i}: {int(his[i])}")
```

Hasil:

```
Intensitas 0: 13
Intensitas 20: 68855
Intensitas 40: 55855
Intensitas 60: 43236
Intensitas 80: 44681
Intensitas 100: 31005
Intensitas 120: 18147
Intensitas 140: 10548
Intensitas 160: 279684
Intensitas 180: 27
Intensitas 200: 0
Intensitas 220: 0
Intensitas 240: 0
```

Perhitungan histogram dalam Numpy:

```
img = cv.imread('home.jpg', flags)

hist, bins = np.histogram(img.ravel(), histSize, (ranges))
```

Dalam OpenCV, *flags* pada fungsi **cv.imread()** memiliki tiga nilai: nilai 1 mewakili **cv.IMREAD_COLOR**, nilai 0 mewakili **cv.IMREAD_GRAYSCALE**, dan nilai -1 mewakili **cv.IMREAD_UNCHANGED** di mana citra dibaca apa adanya. Berikut contoh perhitungan histogram keabuan dengan Numpy:

```
import cv2 as cv
import numpy as np

# Baca gambar dalam mode grayscale
gambar = cv.imread("IMG_9549.jpg", 0)

# Hitung histogram
his, bins = np.histogram(gambar.ravel(), 256, (0, 256))
# Ravel untuk mengubah array berdimensi banyak (2D/3D) menjadi array 1D

# Cetak hanya intensitas kelipatan 20
for i in range(0, 256, 20):
    print(f"Intensitas {i}: {his[i]} piksel.")
```

2.4 Plot Histogram

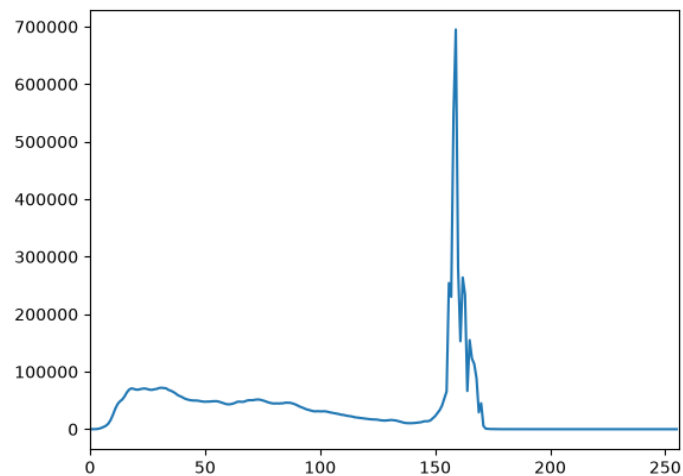
Plot histogram dapat dilakukan dengan menggunakan *library* Matplotlib maupun OpenCV. Pada Matplotlib, histogram ditampilkan menggunakan fungsi **matplotlib.pyplot.hist()**. Contoh: Pada OpenCV, histogram diperoleh melalui fungsi **cv.calcHist()**. Hasil perhitungan dari fungsi ini dapat langsung divisualisasikan dalam bentuk grafik histogram. Berikut contoh kode program plot histogram menggunakan OpenCV:

```
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt

gambar = cv.imread("IMG_9549.jpg", 0)
histo = cv.calcHist([gambar], [0], None, [256], [0,256])

plt.plot(histo)
plt.xlim([0,256])
plt.show()
```

Hasil Output:



Gambar 1. Output Plot Histogram menggunakan OpenCV

2.5 Histogram Stretching

Contrast stretching atau histogram stretching adalah salah satu teknik dalam pengolahan citra digital yang digunakan untuk meningkatkan kualitas citra, khususnya dari segi kontras. Teknik ini bekerja dengan cara meregangkan (*stretching*) distribusi nilai intensitas piksel, sehingga

selisih antara nilai intensitas minimum dan maksimum pada citra menjadi lebih besar. Pada citra dengan kontras rendah, histogram cenderung mengumpul pada satu rentang nilai keabuan tertentu, sehingga perbedaan antara area gelap dan area terang pada citra tidak terlihat jelas. Histogram stretching bertujuan memetakan ulang nilai intensitas tersebut agar menyebar ke seluruh rentang gray level, umumnya dari 0 sampai 255 pada citra grayscale. Histogram yang ideal adalah histogram dengan nilai keabuan yang tersebar secara merata pada seluruh rentang tersebut.

Rumus yang digunakan pada histogram stretching adalah sebagai berikut.

$$g(x, y) = \left[\frac{f(x, y) - f_{\min}}{f_{\max} - f_{\min}} \right] \times 255$$

Keterangan:

- $f(x, y)$: nilai piksel awal pada koordinat (x, y) .
- $f_{\min}$: nilai piksel terkecil (minimum) pada citra.
- $f_{\max}$: nilai piksel terbesar (maksimum) pada citra.
- $g(x, y)$: nilai piksel hasil stretching.

Melalui pemetaan tersebut, nilai intensitas terendah pada citra akan dipetakan mendekati 0, sedangkan nilai intensitas tertinggi akan dipetakan mendekati 255, sehingga rentang kontras citra menjadi lebih lebar. Pada praktiknya, histogram stretching dapat diimplementasikan menggunakan fungsi distribusi kumulatif (*cumulative distribution function/CDF*) dari histogram citra, kemudian nilai CDF tersebut dinormalisasi ke rentang 0–255 dan digunakan sebagai pemetaan nilai piksel citra hasil. Berikut contoh program untuk melakukan histogram stretching terhadap citra ‘low.jpg’.

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

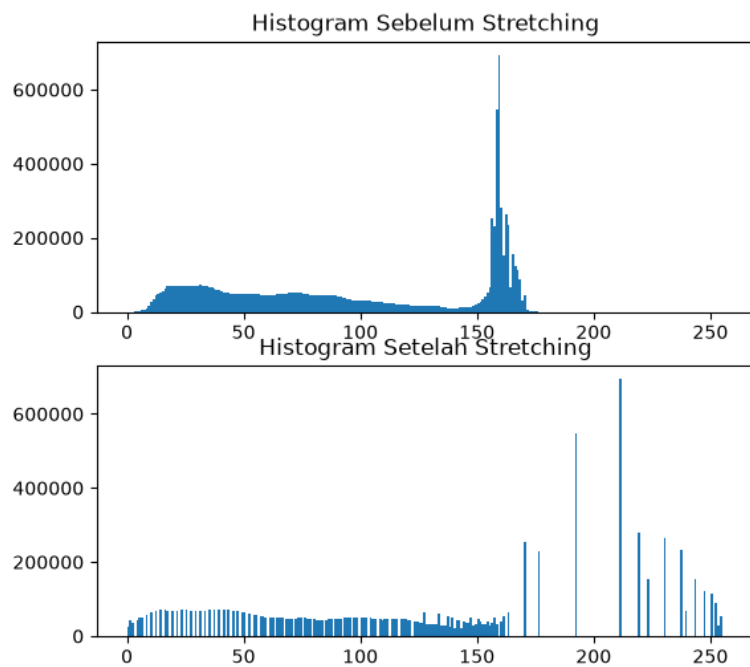
img = cv.imread('IMG_9549.jpg', 0)
hist, bins = np.histogram(img.flatten(), 256, [0, 256])
cdf = hist.cumsum()

# Proses stretching melalui normalisasi CDF
cdf_m = np.ma.masked_equal(cdf, 0)
cdf_m = (cdf_m - cdf_m.min()) * 255 / (cdf_m.max() - cdf_m.min())
cdf = np.ma.filled(cdf_m, 0).astype('uint8')
img2 = cdf[img]
```

```
plt.subplot(2, 1, 1)
plt.hist(img.flatten(), 256, [0, 256])
plt.title('Histogram Sebelum Stretching')

plt.subplot(2, 1, 2)
plt.hist(img2.flatten(), 256, [0, 256])
plt.title('Histogram Setelah Stretching')
plt.show()
```

Hasil Output:



Gambar 2. Output Histogram Stretching

2.6 Histogram Equalization

Histogram equalization adalah teknik peningkatan kontras citra dengan cara meratakan distribusi histogram. Berbeda dengan histogram stretching yang hanya bergantung pada nilai intensitas minimum dan maksimum, histogram equalization mempertimbangkan frekuensi kemunculan setiap tingkat keabuan (gray level) pada citra, sehingga distribusi nilai derajat keabuan pada citra dapat dibuat lebih merata.

Beberapa sifat dari histogram equalization adalah sebagai berikut.

- Gray level yang sering muncul akan dijarangkan (diregangkan) jaraknya terhadap gray level sebelumnya.
- Gray level yang jarang muncul akan dirapatkan jaraknya terhadap gray level sebelumnya.
- Histogram hasil equalization memanfaatkan rentang nilai keabuan secara maksimal, sehingga pasti mencapai nilai keabuan maksimal (misalnya 255).

Untuk dapat melakukan histogram equalization, diperlukan fungsi distribusi kumulatif (CDF) yang merupakan kumulatif dari histogram citra, yang digunakan untuk memetakan ulang nilai intensitas piksel. Pada OpenCV, proses histogram equalization dapat dilakukan secara otomatis melalui fungsi berikut.

```
dst = cv.equalizeHist(src)
```

Penerapan histogram equalization mampu meningkatkan kontras citra secara keseluruhan dengan mendistribusikan nilai piksel secara lebih merata ke seluruh rentang tingkat keabuan, meskipun pada beberapa bagian citra dapat terjadi peningkatan kontras yang tidak merata. Berikut contoh program untuk melakukan histogram equalization terhadap citra 'low.jpg'.

```
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt

# Membaca gambar dalam mode grayscale
img = cv.imread('IMG_9549.jpg', 0)

# Proses histogram equalization
equ = cv.equalizeHist(img)

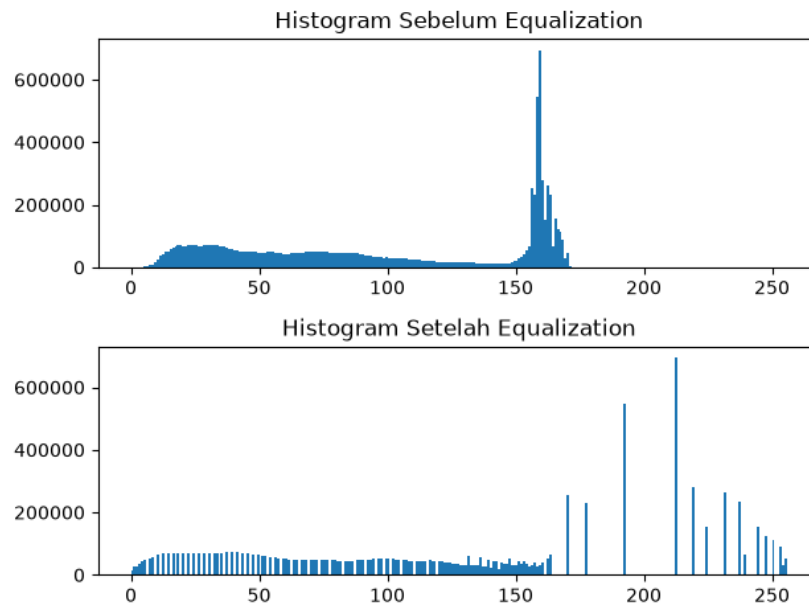
# Menyimpan hasil gambar (opsional, tetap disimpan seperti kode awal)
res = np.hstack((img, equ))
cv.imwrite('res.jpg', res)

# Menampilkan histogram sebelum dan sesudah equalization
plt.subplot(2, 1, 1)
plt.hist(img.ravel(), 256, [0, 256])
plt.title('Histogram Sebelum Equalization')

plt.subplot(2, 1, 2)
plt.hist(equ.ravel(), 256, [0, 256])
plt.title('Histogram Setelah Equalization')
```

```
plt.tight_layout()
plt.show()
```

Hasil Output:



Gambar 3. Output Histogram Equalization

2.7 Histogram Warna

Selain diterapkan pada citra grayscale, konsep histogram juga dapat diterapkan pada citra berwarna melalui histogram warna. Komposisi warna pada suatu citra dapat direpresentasikan dalam bentuk histogram, yaitu histogram yang menggambarkan distribusi jumlah piksel untuk setiap intensitas warna pada citra. Untuk membentuk histogram warna, nilai warna dikuantisasi menjadi beberapa level diskret, kemudian untuk setiap level tersebut dihitung jumlah piksel yang nilainya sesuai.

Berbeda dengan citra grayscale yang hanya memiliki satu kanal intensitas, citra berwarna pada model RGB tersusun atas tiga komponen warna utama, yaitu Red (R), Green (G), dan Blue (B), yang masing-masing memiliki rentang nilai intensitas dari 0 hingga 255 pada citra 8-bit. Karena itu, satu citra berwarna akan menghasilkan tiga histogram, yaitu histogram untuk kanal merah, hijau, dan biru, yang masing-masing dapat dianalisis untuk mengetahui karakteristik warna citra, seperti dominasi warna tertentu maupun pola sebaran intensitas warnanya.

Pada praktiknya menggunakan Python, histogram warna diperoleh dengan membaca citra berwarna menggunakan OpenCV, kemudian citra tersebut dipisahkan ke dalam tiga kanal

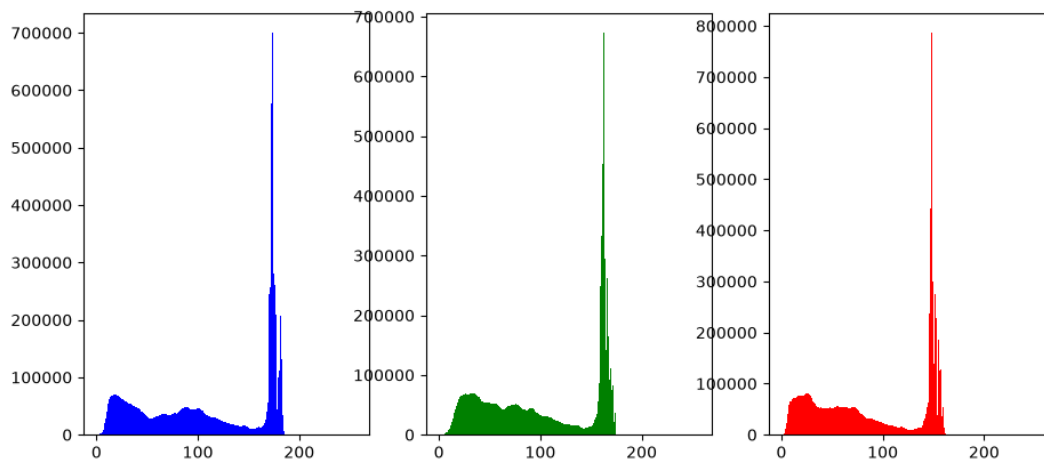
warnanya. Untuk setiap kanal, dihitung distribusi nilai intensitas pikselnya dan divisualisasikan dalam bentuk histogram menggunakan Matplotlib. Berikut contoh programnya.

```
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('IMG_9549.jpg', cv.IMREAD_COLOR)

plt.subplot(1, 3, 1)
plt.hist(img[:, :, 0].ravel(), 256, [0, 256], color='b')
plt.subplot(1, 3, 2)
plt.hist(img[:, :, 1].ravel(), 256, [0, 256], color='g')
plt.subplot(1, 3, 3)
plt.hist(img[:, :, 2].ravel(), 256, [0, 256], color='r')
plt.show()
```

Hasil Output:



Gambar 4. Output Histogram Warna

2.8 Pembentukan dan Interpretasi Histogram Warna

Histogram warna dibentuk dengan menghitung jumlah piksel pada setiap tingkat intensitas untuk masing-masing kanal warna secara terpisah. Secara visual, histogram warna direpresentasikan dalam bentuk grafik dua dimensi, di mana sumbu horizontal menunjukkan nilai intensitas warna (0–255), sedangkan sumbu vertikal menunjukkan jumlah piksel yang memiliki nilai intensitas tersebut. Dengan demikian, satu citra berwarna pada model RGB akan menghasilkan tiga grafik histogram yang masing-masing mewakili kanal merah, hijau, dan biru.

Melalui histogram warna dapat diperoleh berbagai informasi penting mengenai citra, antara lain dominasi warna tertentu, sebaran intensitas warna, serta indikasi tingkat kontras warna. Histogram yang terkonsentrasi pada nilai intensitas rendah menunjukkan citra cenderung gelap, sedangkan histogram yang menyebar pada nilai intensitas tinggi menunjukkan citra cenderung terang. Perbedaan bentuk histogram pada kanal R, G, dan B juga menunjukkan perbedaan kontribusi masing-masing warna dalam membentuk citra secara keseluruhan.

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan pembahasan yang telah diuraikan, dapat disimpulkan bahwa histogram citra merupakan representasi grafis yang menggambarkan sebaran frekuensi kemunculan nilai intensitas piksel pada suatu citra. Pada citra grayscale 8 bit, histogram menampilkan distribusi piksel untuk 256 tingkat keabuan dalam rentang 0 sampai 255, di mana sumbu horizontal menunjukkan nilai intensitas dan sumbu vertikal menunjukkan jumlah piksel yang memiliki nilai intensitas tersebut. Perhitungan histogram dapat dilakukan menggunakan bahasa pemrograman Python melalui dua pendekatan, yaitu fungsi `cv.calcHist()` pada library OpenCV dan fungsi `np.histogram()` pada library Numpy, yang hasilnya kemudian dapat divisualisasikan dengan bantuan library Matplotlib menggunakan fungsi `plt.hist()` maupun `plt.plot()`.

Histogram tidak hanya berfungsi sebagai alat visualisasi, tetapi juga bermanfaat untuk menganalisis karakteristik citra secara kuantitatif. Melalui bentuk dan posisi sebaran histogram, dapat diketahui kondisi exposure suatu citra, baik yang tergolong underexposed, properly exposed, maupun overexposed, serta dapat diketahui pula tingkat brightness dan kontras citra yang ditentukan dari lebar rentang bin yang terisi. Untuk memperbaiki kualitas citra yang memiliki kontras rendah, dapat diterapkan teknik histogram stretching, yaitu teknik yang meregangkan nilai intensitas piksel berdasarkan nilai minimum dan maksimum pada citra sehingga rentang intensitasnya menyebar mendekati 0 hingga 255. Selain itu, terdapat pula teknik histogram equalization yang bekerja dengan mempertimbangkan frekuensi kemunculan setiap gray level melalui fungsi distribusi kumulatif (CDF), sehingga distribusi nilai keabuan pada citra menjadi lebih merata dan memanfaatkan rentang intensitas secara maksimal; pada OpenCV, teknik ini dapat diterapkan secara praktis menggunakan fungsi `cv.equalizeHist()`. Lebih lanjut, konsep histogram juga tidak terbatas pada citra grayscale saja, melainkan dapat diterapkan pula pada citra berwarna melalui histogram warna, di mana pada model RGB setiap citra akan menghasilkan tiga histogram terpisah untuk kanal merah, hijau, dan biru yang masing-masing dapat memberikan informasi mengenai dominasi warna dan sebaran intensitas warna pada citra tersebut. Dengan demikian, dapat disimpulkan bahwa histogram citra merupakan alat bantu yang sederhana namun sangat penting dalam pengolahan citra digital, baik untuk keperluan analisis kualitas citra maupun sebagai dasar bagi teknik-teknik perbaikan citra seperti contrast stretching dan histogram equalization.

Perlu ditegaskan bahwa pembahasan teknik contrast stretching dan histogram equalization pada makalah ini menggunakan citra dengan kontras rendah (low contrast) sebagai studi kasus. Pemilihan ini dilakukan secara sengaja, mengingat efek kedua teknik tersebut akan tampak paling signifikan pada citra dengan histogram yang mengumpul sempit, sementara pada citra dengan

kontras normal maupun tinggi, perubahan visual yang dihasilkan relatif tidak terlalu terlihat karena rentang intensitasnya sudah cukup lebar sejak awal.

Dengan demikian, dapat disimpulkan bahwa histogram citra merupakan alat bantu yang sederhana namun sangat penting dalam pengolahan citra digital, baik untuk keperluan analisis kualitas citra maupun sebagai dasar bagi teknik-teknik perbaikan citra seperti contrast stretching dan histogram equalization.

3.2 Saran

Berdasarkan pembahasan pada makalah ini, penulis menyarankan agar pembaca yang ingin mempelajari lebih lanjut mengenai histogram citra melakukan praktik langsung menggunakan bahasa pemrograman Python dengan bantuan library OpenCV, Numpy, dan Matplotlib, sehingga pemahaman konsep yang bersifat teoritis dapat lebih mudah dipahami secara visual dan aplikatif. Penerapan histogram stretching maupun histogram equalization juga sebaiknya disesuaikan dengan karakteristik citra yang akan diolah, mengingat kedua teknik tersebut memiliki prinsip kerja yang berbeda dan dapat memberikan hasil yang berbeda pula, khususnya pada citra dengan kontras rendah atau pencahayaan yang tidak merata. Selain itu, untuk kajian atau penelitian selanjutnya, dapat dilakukan perbandingan yang lebih mendalam antara metode histogram equalization global dengan metode lanjutan seperti Adaptive Histogram Equalization (AHE) atau Contrast Limited Adaptive Histogram Equalization (CLAHE), yang diketahui mampu mengatasi kelemahan histogram equalization global pada citra dengan variasi pencahayaan lokal. Pembahasan mengenai histogram warna dalam makalah ini juga masih terbatas pada model RGB, sehingga kajian lebih lanjut dapat dikembangkan dengan menganalisis histogram pada model warna lain, seperti HSV atau Lab, yang dalam beberapa kasus lebih relevan untuk keperluan analisis warna maupun segmentasi citra. Akhir kata, penulis menyadari bahwa makalah ini masih memiliki keterbatasan, baik dari segi kedalaman pembahasan maupun contoh kasus yang digunakan, sehingga kritik dan saran yang membangun sangat diharapkan guna penyempurnaan makalah ini di kemudian hari.

DAFTAR PUSTAKA

- Gonzalez, R. C., & Woods, R. E. (2008). *Digital Image Processing* (3rd ed.). Prentice Hall.
- Oliver, W. R. (1998). Histogram Stretching or Histogram Equalization in Image Processing. *Microscopy Today*, 6(3), 20–24. <https://doi.org/10.1017/S1551929500066797>
- Mustafa, W. A., & Abdul Kader, M. M. M. (2018). A Review of Histogram Equalization Techniques in Image Enhancement Application. *IOP Conference Series: Journal of Physics: Conference Series*, 1019, 012026. <https://doi.org/10.1088/1742-6596/1019/1/012026>
- Xiong, et al. (2021). Application of Histogram Equalization for Image Enhancement in Corrosion Areas. *Shock and Vibration*, 2021, Article 8883571. <https://doi.org/10.1155/2021/8883571>
- Yousuf, M. A., & Rakib, M. R. H. (2011). An Effective Image Contrast Enhancement Method Using Global Histogram Equalization. *Journal of Scientific Research*, 3(1), 43–50. <https://doi.org/10.3329/JSR.V3I1.5299>